

# Map-Aided Automotive Dead-Reckoning using Rao-Blackwellized Particle Filter

Aleksandr Mikov\*, Roman Voronov<sup>†</sup> and Alex Moschevikin<sup>‡</sup>

Petrozavodsk State University

Email: \*sasha.mikoff@gmail.com, <sup>†</sup>rvtoronov@petrsu.ru, <sup>‡</sup>alexmou@lab127.karelia.ru

**Abstract**—The paper presents the dead-reckoning algorithm for the vehicles. The algorithm is based on Rao-Blackwellized Particle Filter and operates in global coordinates (latitude, longitude and altitude). The key feature is the usage of buildings' footprints and road networks for position aiding. There are no restrictions on off-road driving. The algorithm fuses the data from low-cost IMU, odometer, road and buildings' maps. The system state consists of vehicle's position, its velocity and attitude, as well as the sensors' errors.

The algorithm can operate in real-time on a PC with the number of particles up to 10000. This performance is achieved by utilizing the spatial trees for map data requests and error-state covariance propagation.

The algorithm was tested by extensive simulations and real experiments. The average positioning error (APE) in the case of perfectly known road network and buildings' footprints did not exceed 2 meters after 10 minutes of operation in dead-reckoning mode. The maximum error did not exceed 7 meters. In the case of off-road driving and roads' and buildings' representation inaccuracies, the APE after 10 minutes of autonomous operation was  $\approx 12$  meters. The proposed solution allows to aid the odometry error drift and provide lane-level navigation accuracy on a long periods of operation in dead-reckoning mode.

**Index Terms**—vehicle dead-reckoning, inertial sensors, particle filter, rao-blackwellization, navigation

## I. INTRODUCTION

The demand on the accurate positioning [1] for cars, smart city devices, robots and semi-autonomous platforms increased the requirements for both the quality of the sensors and the algorithms that fuse the data.

Today, the inertial navigation systems (INS), global navigation satellite systems (GNSS), radars, lidars, odometers and cameras are used for the vehicle's localization. Each of the sensors has its own noise characteristics that directly affect the positioning quality. There are several options to improve the positioning accuracy:

- 1) Select the higher grade sensors.
- 2) Fuse the data from a number of sensors to compensate their errors.
- 3) Use the algorithms for fusion of implicit, or external information, that can potentially improve the positioning accuracy.

The first option is the most compelling: by reducing the noise at the input of the system, the noise of its output will be reduced. The drawback is obvious – it is associated with a significant price increase. To use the second option it is necessary to deploy an additional hardware and maintain it. It

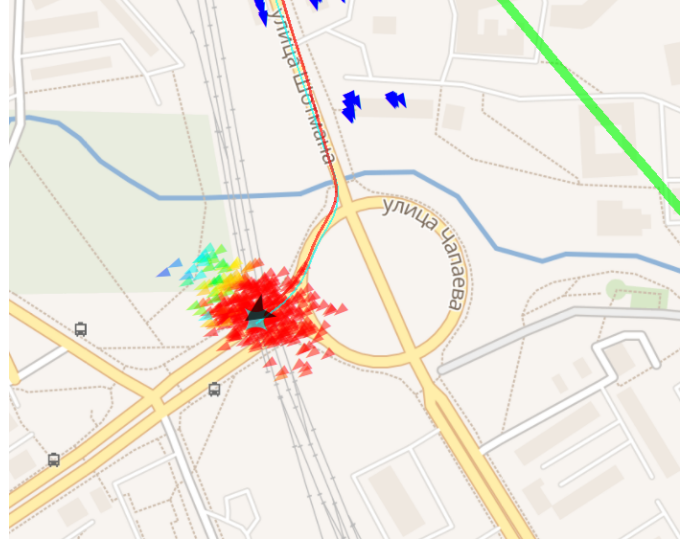


Fig. 1: The algorithm performance in dead-reckoning mode. The green line is the EKF solution, the red line – the Rao-Blackwellized Particle Filter (RBPF) solution, the blue line is the smoothed GNSS solution (ground truth). The black arrow is the weighted position solution from RBPF, the blue rectangle is the current GNSS solution. The distributed small arrows – hypotheses about the possible positions from RBPF, their colors reflect the relative probability of the hypothesis. Blue – unlikely, red – very likely, dark blue – the hypothesis zeroed by buildings' footprints. The time since the GNSS corrections were switched off is 810 seconds.

can also be costly. The third option allows to leave the system "as is" in terms of hardware, and use the external information on the algorithm side. The challenge of the latter option is the art of algorithm development.

In this work, we argue for the third approach. It is shown that low-cost inertial sensors can serve as a basis for the high-quality positioning system. It is possible by the position aiding using the publicly available roads' and buildings' maps. This information effectively decreases the position uncertainty and limits the growth of the positioning error.

The accelerometers and gyroscopes have long been used for positioning in dead-reckoning mode [2]. High-grade sensors can provide an accurate positioning on aircraft or boats, where their usage is plausible. At the same time, the measurement errors of the low-cost inertial sensors limit their widespread

use for vehicles' positioning in dead-reckoning mode.

The error sources for INS can be divided into two groups: the stochastic and the systematic (or instrumental). The first group includes the white noise, bias instability and random walk. The effect of these errors on the positioning accuracy is difficult to eliminate without external corrections. The second group includes the scale factors, axes non-orthogonality and the misalignment between the gyroscope and accelerometer triads. These errors might be compensated using the calibration procedure.

As an illustrative example let's compare the tactical and low-cost MEMS gyroscope. Consider a tactical grade gyroscope [3] with a bias instability ( $BI$ ) equal to  $0.9^\circ/hr$ , and angle random walk ( $ARW$ ) equal to  $0.2^\circ/\sqrt{hr}$ . Let's further assume that the system works in dead-reckoning mode for 3 minutes (180 seconds). Then the root mean square deviation (RMSD) of the coordinate caused by the gyroscope bias instability is approximately equal to 12 meters (the correlation time of the bias instability  $T_c$  is equal to 600 seconds) [4]:

$$\sigma_{BI} \approx g \cdot BI \frac{1}{\sqrt{126T_c}} t^{3.5} \approx 9.8 \frac{0.9}{3600} \frac{\pi}{180} \frac{1}{\sqrt{12600}} 180^{3.5} \approx 12m. \quad (1)$$

RMSD of the coordinate error after 180 seconds caused by angle random walk is:

$$\sigma_{ARW} \approx \frac{g \cdot ARW}{2\sqrt{5}} t^{2.5} \approx \frac{9.8 \cdot \frac{0.2}{60} \frac{\pi}{180}}{2\sqrt{5}} 180^{2.5} \approx 55m. \quad (2)$$

The RMSD of the position, induced by stochastic errors of the low-cost MEMS-sensors, is higher and approximately equal to  $\sigma_{BI} \approx 74m$  (with  $BI = 5.5^\circ/hr$ ) and  $\sigma_{ARW} \approx 83m$  (with  $0.3^\circ/\sqrt{hr}$ ). After three minutes of operation in dead-reckoning mode one can judge only about the estimated area of the object, but not about the estimated position. The range of the potential vehicle position exceeds hundreds of meters. The MEMS sensors are much worse than the tactical grade sensors, but much more cheap and therefore widely used in consumer devices.

The positioning error induced by the instrumental errors of the MEMS sensors is at the same level [4], as for the stochastic errors. Its actual magnitude depends on the deviation of real sensor scale factors and non-orthogonality from their nominal values. [5].

Since the RMSD of the accumulated error in dead-reckoning mode is quite high, INS solution is usually aided by the information from other positioning systems or correction sources. [6], [7] show in practice that the use of the odometer allows to reduce the RMSD of the positioning error up to:

$$\sigma_p(t) \approx \bar{v} \sqrt{\left( BI \cdot t^2 / (2\sqrt{2} \ln(2)/\pi) \right)^2 + (2t^{1.5} \cdot ARW/3)^2}, \quad (3)$$

where  $\bar{v}$  is the average motion speed of the vehicle. After the substitution of  $BI$  and  $ARW$  values and assuming that the average speed is equal to  $20 \frac{m}{s}$  the RMSD of the coordinate error by the 180th second will be  $\approx 3m$  (or  $\approx 13m$  for low-cost MEMS IMU) and reduced significantly compared to the INS only solution.

The article focuses on the problem of data fusion from:

- 1) INS and GNSS;
- 2) odometry;
- 3) roadmaps and surrounding buildings' footprints. An example of the map used in the fusion process is shown on Fig. 1.

Due to the different nature of the correction sources, the most effective fusion solution can be obtained using the combination of the Kalman filters (KF) with the particle filter (PF). Such a technique is known as Rao-Blackwellized particle filter. There are two key features of the algorithm: the use of rough buildings' footprints for corrections and no restrictions on off-road driving. The motion of each particle in the RBPF corresponds to the hypothesis about the position of the vehicle. The trajectory of the particle is calculated according to the state propagation functions using the accelerometer and gyroscope measurements, this trajectory is being corrected using GNSS, odometry or map-matching measurements. The number of particles corresponds to the size of the Kalman filter bank.

The rest of the paper is organized as follows. Section II describes the state-of-the-art methods for positioning, the first subsection II-A is devoted to Kalman filters, the second II-B – to the Particle filters. Section III describes the developed algorithm: the system model III-A, measurement models III-B and the usage of roadmap III-C and buildings' footprints III-D information. Then, in subsection III-E, the algorithm pseudocode is presented. Section IV describes the simulation model IV-A, the datasets, used for testing IV-B, the processing methods IV-C and analysis of the results IV-D. Section V concludes the paper and outlines the future research goals.

## II. FILTERING METHODS

### A. Kalman-based filters

The widely used tool for data fusion from different sensor sources is the Kalman Filter (KF). Since the linear Kalman filter can work with linear systems only in the case of nonlinear dynamics or measurement functions, they have to be linearized at the operating point. This point is usually selected as the current estimate from the filter itself. Such a filter is called the extended Kalman Filter (Extended Kalman Filter – EKF). EKF demonstrated its reliability and performance starting from first space programs (Apollo, Soyuz, Vostok) to modern problems of positioning and tracking [8], [9].

One of the problems of EKF is the uncertainty underestimation. The mathematical and physical explanation for this phenomenon was discussed in work [10], devoted to the problems of simultaneous localization and mapping. It was shown that from the math point of view, the observability matrix is not full-rank, and therefore the system is not fully observable. The physical meaning was revealed through the basis of the observability matrix. One of its basis vectors is the rotation vector around the vertical. It means that the rotation around Z-global coordinate can not be observed, but it is what actually happens in the filter and leads to its inconsistency.

To mitigate this problem the observability constraints were presented by the authors.

Another drawback of EKF is the assumption that the underlying distributions are normal. It has been noted many times that the true form of the probability density function of the position of the moving object is not elliptical, but rather resembles the "banana"-shaped distribution 2. As the time of operation in dead-reckoning mode increases, the hypothesis about the underlying Gaussian distribution ceases to be consistent, that leads to unstable filter operation.

In [11] the solution to this problem was proposed: the use of Lie algebra machinery. It was noted, that the errors of the pose are normally distributed in the Lie algebra space. These errors, parameterized using the Lie algebras, can be exponentially mapped onto a Lie group space and the original Gaussian distribution will have the form of a "banana-shaped" distribution. This observation is shown on Fig. 2. Subsequently, in [12], [13] the theory of the invariant EKF was presented, that is based on the Lie groups theory in the  $SE_2(3)$  space. The main benefits of the proposed approach is the potential full autonomy of the orientation and coordinate errors – the theoretical independence from the nominal state. Its results in the improved filter convergence under the heading misalignment and enhanced filter stability for simultaneous localization and mapping problems.

In work [14] the efficiency of the invariant EKF for INS, odometry and GNSS fusion was studied. The right and left error invariants were analyzed. The existence of two variants for error definition is explained by the non-commutativity of matrix multiplication, and, accordingly, composition rules using the Lie groups [15]. It was found that the left error definition is more suitable for INS-GNSS fusion. In this case the dynamics of the errors does not depend on the trajectory. The important disclosure of the paper is the fact that for INS-odometry fusion the measurement model is not independent of the trajectory. It was experimentally confirmed that the use of the  $SE_2(3)$  group in comparison with the use of the traditional error definition in  $SO(3)$  does not automatically lead to better results. This means that the choice problem for an optimal fusion scheme remains. In the proposed algorithm we will use  $SO(3)$  error parametrization, since it is well studied and all the measurements and state-transition functions can be efficiently derived.

An alternative to EKF with non-linear dynamics or the measurements is the UKF (Unscented KF – UKF). It also tracks the first two moments of the distribution, but calculates them using the statistical methods. The filter picks several points around the mean and propagates them using the given function. After that the mean and covariance are numerically calculated. There exist several extensions for UKF when the analytical derivation of the Jacobians is not necessary – they can be estimated numerically. This greatly simplifies the prototyping speed. The main drawback of the filter is its higher computational load compared to EKF and the problem of parameters' selection for the sigma-points [16].

It should be noted that for practical implementations of the

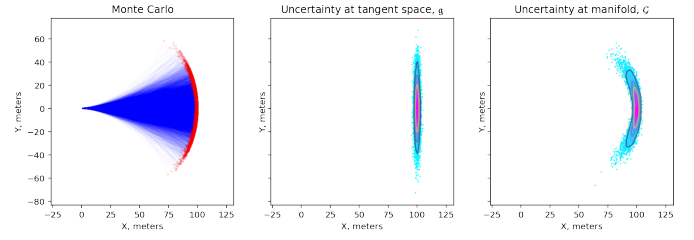


Fig. 2: Uncertainty of the vehicle's position after making 100 steps from the point (0, 0) in the direction of X axis. The standard deviation of the heading angle measurement at each step was set to 0.03 radians; there was no measurement noise for the step length measurement. Left – the result of the simulation using Monte-Carlo methods. Middle – the result of sampling from a multivariate normal distribution in Lie algebra space, obtained by extrapolating the initial covariance matrix with specified noise parameters. Right – samples from the central middle plot after exponential mapping onto the Lie group space.

KF algorithms the error-state definition is preferable. In such a case the errors of the observed values are estimated in the filter instead of the values itself [17]. The errors are defined as:  $\delta \mathbf{x} = \mathbf{x} - \hat{\mathbf{x}}$ , where  $\delta \mathbf{x}$  – the error-state,  $\mathbf{x}$  – the true state,  $\hat{\mathbf{x}}$  – the nominal, or estimated state. The reason for this is twofold. First, the dynamic of the error is lower (often linear) than the dynamics of the nominal state (since the main dynamics is already taken into the account when modelling the nominal state). Second, the filter covariance matrix can be propagated less frequently, since its dynamics is closer to the linear one. This allows to significantly reduce the computational load of the filter.

The aim of this work is to develop the robust and efficient algorithm for INS, odometry, roadmap and buildings' footprints data fusion. The pure KF is not applicable there. The existing approaches [18], [19] for KF are based on the projection of the current position estimate onto the roadmap and generation of the virtual measurement. This approach, however, is not fully reliable and can work only if initial position and heading errors are low.

### B. Particle filters

The particle filter is a universal tool for filtering the nonlinear systems. The PF is not the parametric one and allows to work with any probability distribution, thanks to resampling and particle weighting procedures at the prediction and correction steps. The posterior density in the filter is approximated by a finite number of particles, or stochastic grid points. According to the law of large numbers, with an increase of the number of particles, the approximation of the probability density function (PDF) converges to the true value [20]. Therefore, the quality of the filter's estimates depends on the number of particles. Due to the "curse of dimensionality", the direct application of the particle filter for high-dimensional problems is complicated – the number of

particles required for an adequate PDF representation grows exponentially with the problem dimensionality. An illustration to the "curse of dimensionality" is shown on Fig. 3.

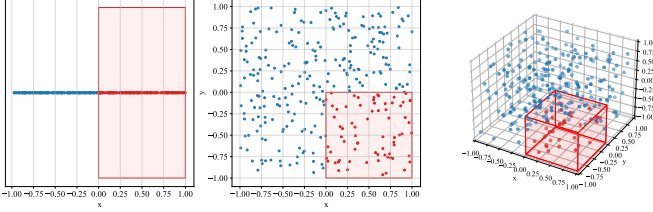


Fig. 3: The "curse of dimensionality" problem. In order to properly describe the state of the system using the PF, the number of particles should be doubled each time the dimension of the problem is increased by one.

---

#### Algorithm 1 Particle filter

---

- 1) Filter initialization. Sample the particles  $\{x_0^{(i)}\}_{i=1}^N \sim p_{x_0}$  and initial weights  $\{w_0^{(i)}\}_{i=1}^N = \frac{1}{N}$  for  $i = 1, \dots, N$ .  $N$  – the number of particles.
- 2) Prediction step. For each particle ( $i$ ) predict its state  $x_{k|k-1}^{(i)} \sim p(x_k|u_t, x_{k-1}^{(i)})$ .
- 3) Correction step. For each particle ( $i$ ) update its weight according to the measurement likelihood  $y$ :  $w_{k|k}^{(i)} = p(y_k|x_k^{(i)}) \cdot w_k^{(i)}$ .
- 4) Normalize the weights:

$$w_{k|k}^{(i)} = \frac{w_k^{(i)}}{\sum_{j=1}^N w_{k|k}^{(j)}}. \quad (4)$$

- 5) Make the resampling, if needed: duplicate the particles proportional to their weights  $w_{k|k}^{(i)}$  and build the new sets  $\{x_k^{(i)}\}_{i=1}^N, \{w_k^{(i)}\}_{i=1}^N = \frac{1}{N}$ .
- 

The general PF scheme can be described by Algorithm 1. The key step is the resampling procedure. It allows to redistribute the particles according to the posterior. In [21] noted that resampling procedure is a probabilistic implementation of the Darwinian idea "the survival of the fittest". The need for resampling is determined by the number of "effective" particles – their amount decreases as the particles deplete. It is shown in [22], that the number of effective particles can be well approximated by the expression  $\hat{N}_{eff} = \frac{1}{\sum_{(i)} (w^{(i)})^2}$ .

To solve the problem associated with the "curse of dimensionality" several approaches exists in the literature. They enable the use of the PF in the spaces of the larger dimensions.

One of such techniques is the Rao-Blackwellization [23], [24]. The state of the system is split into the two parts, the first part  $\mathbf{x}^n$  employs the particles to represent the posterior and the second part  $\mathbf{x}^l$  – parametric distributions. Such a filter is called

Rao-Blackwellized Particle Filter (RBPF). The posterior of the combined vector  $\mathbf{x} = [\mathbf{x}^n, \mathbf{x}^l]^T$  can be represented as:

$$p(\mathbf{x}|\mathbf{y}) = p(\mathbf{x}^n|\mathbf{x}^l, \mathbf{y})p(\mathbf{x}^l|\mathbf{y}), \quad (5)$$

where  $p(\mathbf{x}^n|\mathbf{y})$  is approximated by a finite particle set, and integration by parts allows to get rid of the component  $\mathbf{x}^l$ .

In general, the model of such a system can be represented as [24]:

$$\mathbf{x}_{k+1}^n = \mathbf{f}_k^n(\mathbf{x}_k^n) + \mathbf{F}_k^n(\mathbf{x}_k^n)\mathbf{x}_k^l + \mathbf{G}_k^n(\mathbf{x}_k^n)\omega_k^n, \quad (6a)$$

$$\mathbf{x}_{k+1}^l = \mathbf{f}_k^l(\mathbf{x}_k^n) + \mathbf{F}_k^l(\mathbf{x}_k^n)\mathbf{x}_k^l + \mathbf{G}_k^l(\mathbf{x}_k^n)\omega_k^l, \quad (6b)$$

$$\mathbf{y}_k = \mathbf{h}_k(\mathbf{x}_k^n) + \mathbf{H}_k(\mathbf{x}_k^n)\mathbf{x}_k^l + \mathbf{e}_k, \quad (6c)$$

where  $\mathbf{f}$  – vector function, that describes the system dynamics and depends on  $\mathbf{x}^n$ ,  $\mathbf{F}$ ,  $\mathbf{H}$  and  $\mathbf{G}$  – arbitrary matrices,  $\mathbf{h}$  – measurement function. The noises included in the model are assumed to be normally distributed with zero mean.

In paper [25] it was shown that RBPF filtering problem can be implemented using the Kalman filter bank technique. In this case, the state vector does not need to be explicitly divided into two parts, that are propagated separately, the dynamic and the measurement model remain the same as in the case of the EKF. From time to time it is necessary to "mix" the state, represented by the particles and state, represented by the parametric distributions. To do so the "virtual" measurements from the particles' uncertainties are sampled and then fed back to the Kalman filters as the corrections.

Work [26] overviews the use of particle filters and their variants for the vehicles' positioning. It is pointed out that PF is the main method for working with multimodal distributions that arise when the road maps and odometry are fused together.

It should be noted, that the situation when the vehicle position is restricted to the roadmap introduces a significant limitation – the inability of positioning in the areas where an exact roadmap is absent, or it is not feasible to build it (parking lots, courtyards, and so on). The usage of the buildings' footprints in dense urban environment (see Fig. 1) allows to overcome this issue. As a result, it is possible to provide the robust positioning in the areas, where the roadmap is absent or has poor quality. The proposed algorithm can deal with these scenarios by utilizing buildings' footprints data for corrections.

### III. ALGORITHM DESCRIPTION

Let us recall the main idea of RBPF, or KFB (Kalman Filter Bank): the state of the system is divided into two parts, the first part uses the particles to represent the underlying distribution, the second part uses the parametric distributions' moments. In the following algorithm, the errors in the north and east local coordinates are represented using the particles. All the other states are represented using the multivariate normal distribution, which parameters are estimated in the KF. We will use the terms RBPF and KFB interchangeably, since each particle is the instance of the KF.

### A. System model

The system model consists of two vectors: the nominal state and the error-state. The true state  $\mathbf{x}$  is defined as the composition of the nominal state  $\hat{\mathbf{x}}$  and perturbation, or error  $\delta\mathbf{x}$ . The proposed algorithm estimates the following model parameters:

- 1)  $\mathbf{R}_b^n$  – the rotation matrix from body frame, associated with INS, to local coordinate plane  $(N, E, D)$ .
- 2)  $\mathbf{v}_{eb}^n$  – the INS velocity relative to the inertial coordinate system associated with the Earth, resolved in  $(N, E, D)$  frame.
- 3)  $\mathbf{p} = [\phi, \lambda, h]^T$  – latitude, longitude and altitude.
- 4)  $\mathbf{b}_a$  – the accelerometer biases.
- 5)  $\mathbf{b}_g$  – the gyroscope biases.
- 6)  $s$  – odometer scale factor.
- 7)  $t_{IG}$  – the GNSS solution delay, or time lag.

In the algorithm configuration it is possible to define the lever arms: the GNSS receiver position and the center of the rear axle of the vehicle with respect to the INS. They are named as  $\mathbf{l}_{IG}^b$  and  $\mathbf{l}_{IR}$ , respectively.

The error-state dynamics can be obtained by linearization of the true state dynamics at the operating point (usually the current estimate  $\hat{\mathbf{x}}$ ):

$$\delta\mathbf{x}_{k+1|k} = \mathbf{f}(\mathbf{x}, \mathbf{u}) - \mathbf{f}(\hat{\mathbf{x}}, \mathbf{u}) = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \Big|_{\hat{\mathbf{x}}_{k|k}} \delta\mathbf{x}_{k|k}. \quad (7)$$

Then the system state vector has the following form:

$$\delta\mathbf{x} = [\delta\epsilon, \delta\mathbf{v}, \delta\mathbf{p}, \delta\mathbf{b}_a, \delta\mathbf{b}_g, \delta s, \delta t_{IG}]^T. \quad (8)$$

The state transition matrix  $\mathbf{F}$  for such a system is well known and was derived in the literature. The corresponding equations for  $\mathbf{F}$  are presented, for example, in [27, pp. 584-589]. The propagation of the nominal state is just the integration of the accelerometer and gyroscope measurements. Then the state vector (8) and its covariance matrix are being observed in KFB.

### B. Measurement models

1) *GNSS measurement model*: since the time delay of the GNSS solution relative to the INS solution was added to the state vector, the corresponding measurement function should be presented:

$$\delta\mathbf{z}_G^n = \mathbf{T}_p^{r(n)} \cdot (\mathbf{p}_G - \mathbf{p}) + t_{IG} \cdot \mathbf{v}_{eb}^n + \mathbf{R}_b^n \cdot \mathbf{l}_{IG}^b, \quad (9)$$

where  $\mathbf{T}_p^{r(n)}$  – transformation matrix of the coordinate error from the curvilinear coordinate system to the local tangent plane  $(N, E, D)$ .

The measurement matrix  $H$  is calculated as the Jacobian of function  $\delta\mathbf{z}_G^n$  w.r.t. the state vector  $\delta\mathbf{x}$  at  $\hat{\mathbf{x}}$ . Therefore,

$$H_k = \frac{\partial \delta\mathbf{z}}{\partial \mathbf{x}} \Big|_{\mathbf{x}=\hat{\mathbf{x}}}.$$

2) *Odometer measurement model*: taking into account the constraints on zero vertical and lateral velocities of the vehicle, we get the following form of the odometer's measurement function:

$$\delta\mathbf{z}_o^n = [v_{odo} \cdot (1 - \delta s), 0, 0]^T - \mathbf{R}_n^b \cdot \mathbf{v}_{eb}^n - \boldsymbol{\omega}_{ib}^b \times \mathbf{l}_{IR}, \quad (10)$$

where  $v_{odo}$  – the vehicle speed, as reported by odometer,  $\delta s$  – odometer scale factor error,  $\boldsymbol{\omega}_{ib}^b$  – gyroscope measurements,  $\times$  – vector cross product. The observation matrix is calculated in the same way as for GNSS measurements.

### C. Corrections using roadmap

The road network is presented as a graph. The classification of the road network usage schemes for corrections was proposed in [28] and reproduced here for convenience:

- 1) The generation of virtual measurements – the idea's essence is to weight the particles proportionally to the distance to the nearest edge of the graph.
- 2) The space constraints – on the prediction step, the particles, that are outside the road "corridors" are eliminated.
- 3) Manifold filtration – the state of the particle is described by the roadmap's edge identifier and the distance relative to the head of the edge.

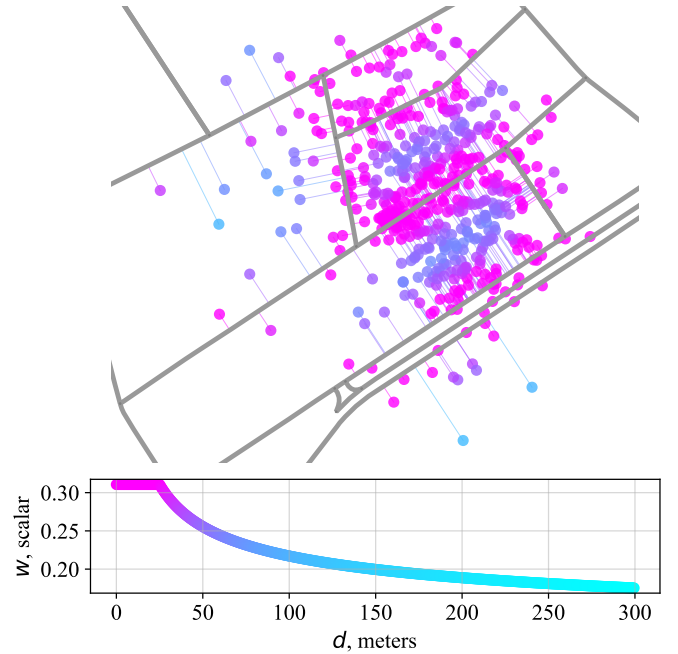


Fig. 4: Corrections using the roadmap. The upper image is the distribution of hypotheses about the vehicle's position. Each point is a hypothesis, thick gray lines – the edges of the roadmap, thin gray segments – the segments connecting the position's hypothesis and the nearest point of the roadmap. Bottom image – particles weighing function (11), that depends on the distance to the nearest edge of the roadmap,  $d_{min} = 25$  m.

The first option was chosen for the developed algorithm by the authors. The weighing function was synthesized in a way to allow the position of the vehicle to be not only on the roadmap, but also in a large area around it. The road edges were packed into the spatial tree, limiting the time spent for the nearest edge lookup. For weighing the particles the following formula was used:

$$w(d) = \frac{1}{\log(\max(d_{min}, d))}, \quad (11)$$

where  $d$  – the distance from the particle to the nearest edge of the graph. The usage of max function inside the logarithm allows to have the same weight for all particles that are closer than  $d_{min}$  meters to the edges. The logarithm function "softly" updates the particles weights depending on the particle-road distance. The  $d_{min}$  value can be selected empirically, based on the surrounding buildings' density and quality of the roadmap. The lower the roadmap representation quality, the higher  $d_{min}$  value should be preferred. Based on our experiments we propose to use  $d_{min} = 2.5$  m for high-quality road network representation, when there are no missing lanes or road edges.  $d_{min} = 25$  m should be preferred when the road network representation quality is low and a lot of off-road driving is expected.

The example of the particles weighing using  $w(d)$  function is shown on Fig. 4. After weighing according to (11), the particles have likelihoods proportional to the distance to the nearest roadmap's edge.

#### D. Corrections using buildings' footprints

The usage of buildings' footprints for position aiding is implemented in the prediction step of the algorithm. The trajectory of the particle is examined: if it intersects the building's geometry, then the particle weight is zeroed:

$$w_{k+1|k}^{(i)} = \begin{cases} 0, & \text{if } [\mathbf{p}_k^{(i)}, \mathbf{p}_{k+1}^{(i)}] \cap b, \forall b \in \text{Buildings} \\ w_k^{(i)} & \end{cases}. \quad (12)$$

To improve the algorithm performance, the buildings' geometries are stored in the array. The array binds the building's geometry and its bounding box. This allows to pack the building's bounding boxes inside the R-tree and access their geometry in a constant time. The first step here is to check whether the path intersects the bounding box of any building. If so, check whether it intersects the building's geometry.

The example of the corrections using the buildings' footprints is shown on Fig. 5.

#### E. Implementation details

As shown in [25], the state model (6) could be simplified to:

$$\mathbf{x}_{k+1} = \mathbf{F}_k(\mathbf{x}_k^n)\mathbf{x}_k + \mathbf{f}(\mathbf{x}_k^n) + \mathbf{G}_k(\mathbf{x}_k^n)\omega, \quad (18a)$$

$$\mathbf{y}_k = \mathbf{H}_k(\mathbf{x}_k^n)\mathbf{x}_k + \mathbf{h}(\mathbf{x}_k^n) + \mathbf{e}_k. \quad (18b)$$

The developed Rao-Blackwellized Particle Filter in terms of stochastic Kalman filter bank is described by Algorithm 2. The  $-$  and  $+$  symbols are the prior and posterior variables,  $\mathbf{Q}$  –

---

#### Algorithm 2 RBPF algorithm for vehicle map-aided dead-reckoning

---

- 1) Filter initialization. Allocate the Kalman filter bank, initialize the nominal states  $\mathbf{x}_0^{(i)}$ , state covariances  $\mathbf{P}_0^{(i)}$  and initial weights  $w_0^{(i)} = \frac{1}{N}$  for  $i = 1, \dots, N$ .  $N$  – the number of filters in the bank. The next steps are repeated for each filter  $(i)$  from the bank (i.e. each particle).
- 2) Prediction step.
  - On each new inertial measurement (accelerometer and gyroscope), update the nominal states  $\mathbf{R}_b^{n(i)}, \mathbf{v}_{eb}^{n(i)}, \mathbf{p}^{(i)}$ .
  - Check whether the filter  $(i)$  trajectory intersects the nearby buildings' footprints. Update the filter weights according to (12), and normalize the weights according to (4).
  - If the "mixing", or marginalization is needed ( $T_m$  seconds have passed since last "mixing"), then sample the position errors according to:  $\xi^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{P}_{7:8,7:8}^{(i)} + \Xi)$  and correct KF states with sampled  $\xi^{(i)}$  values: use them as the measurements according to (15), (16).
- 3) Correction step. On any measurement arrival propagate the covariance matrix  $\mathbf{P}^{(i)}$  from step  $k-1$  to step  $k$  by time  $\Delta t = t_k - t_{k-1}$ :

$$\mathbf{P}_k^{(i)-} = \mathbf{F}_k \mathbf{P}_{k-1}^{(i)+} \mathbf{F}_k^T + \mathbf{Q}_k^{(i)}. \quad (13)$$

- For odometry or GNSS measurements obtain the measurement innovation  $\delta \mathbf{z}^{n(i)}$  according to (9), (10) and the corresponding measurement matrix  $\mathbf{H}_k^{(i)}$ . Update the filter weights:

$$w_k^{(i)+} \propto \mathcal{N}(\delta \mathbf{z}^{n(i)}; \mathbf{0}, \mathbf{S}^{(i)}) \cdot w_k^{(i)-}. \quad (14)$$

Calculate the error-state vector, correct the nominal state, update the covariance matrix:

$$\delta \mathbf{x}^{(i)} = \mathbf{K}^{(i)} \cdot \delta \mathbf{z}^{n(i)}, \quad (15)$$

$$\mathbf{P}_k^{(i)+} = \mathbf{P}_k^{(i)-} - \mathbf{K}^{(i)} \mathbf{S}^{(i)} \mathbf{K}^{(i)T}, \quad (16)$$

where  $\mathbf{S}^{(i)} = \mathbf{H}_k^{(i)} \mathbf{P}_k^{(i)-} \mathbf{H}_k^{(i)T} + \mathbf{R}_k^{(i)}$ ,  $\mathbf{K}^{(i)} = \mathbf{P}_k^{(i)-} \mathbf{H}_k^{(i)T} \mathbf{S}^{-1}$ .

- Correct the state using the roadmap (with period  $T_{graph}$ ). Find the distance  $d$  from the filter  $(i)$  position estimate to the nearest edge of the graph and update the weight of the filter according to equation (11):

$$w_k^{(i)+} \propto w(d) \cdot w_k^{(i)-}. \quad (17)$$

- After any measurement update, or correction, normalize the weights according to (4).
- 4) If resampling is required ( $N_{eff} < N_{threshold}$ ), then resample the filters from the bank proportionally to their weights.
-

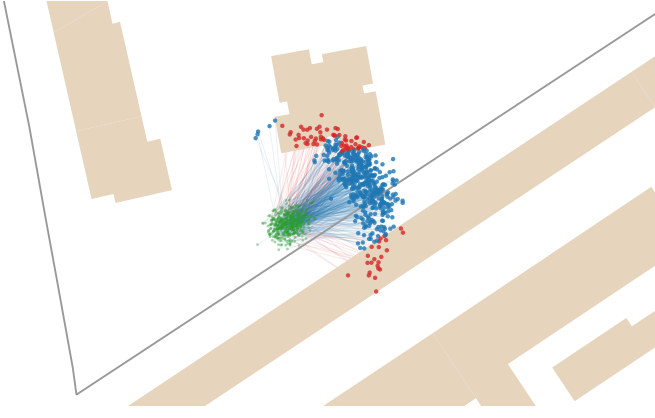


Fig. 5: Corrections using buildings' footprints. Green circles – the particles positions at time  $k$ , blue and red circles – the position propagation at time  $k + 1$ . If  $[\mathbf{p}_k, \mathbf{p}_{k+1}]$  segment intersects the building's geometry, the particle weight is zeroed. The particles which weights were zeroed are marked in red.

process noise,  $\mathbf{R}$  – measurement noise,  $\Xi$  – an additive noise, not described by the model,  $\mathbf{P}_{7:8,7:8}$  – the block sub-matrix of  $\mathbf{P}$ , that consists of expected covariances of position errors in north and east directions.

It can be seen that in this formulation, any EKF-based filter can be easily converted to a stochastic Kalman filter bank. This is especially useful for practical implementations, when it is required to introduce implicit corrective information into the filter or to quickly expand the system with new states.

In practice, it is possible that the majority of the particles, or filters, in the bank have zero weights. This means that the filter diverged and needs to be reinitialized. To handle this problem the EKF runs in parallel with RBPF. If reinitialization is needed, then it is triggered around the current EKF estimate.

#### IV. EXPERIMENTS AND RESULTS

##### A. Simulation model

To extensively test the algorithm the data simulator was developed. It was used to generate the measurements from all the sensors presented on the target vehicle (GNSS receiver, odometer, gyroscope, accelerometer). One cycle of data simulation consisted of the following steps:

- 1) The selection of the start and end points of the route on the road graph.
- 2) The search of the route between start and end points. The route interpolation using the cubic splines.
- 3) The generation of ideal accelerometer, gyroscope, odometer, and GNSS receiver measurements along the route. It was assumed that the vehicle can move only along its forward axis, and therefore the assignment of the orientation angles was not necessary – they were calculated automatically from the linear velocity vector.
- 4) The generation of accelerometer, gyroscope, odometer and GNSS receiver measurement errors. Both the instrumental (scale factor, non-orthogonality of axes, bias,

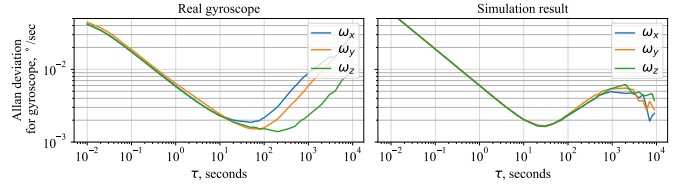


Fig. 6: Allan deviations' plots for real sensor (left) and simulation (right).

misalignment between the triads) and the stochastic errors ( $BI$ ,  $ARW$  and white noise) were modelled.

- 5) The generation of the simulated sensor readings as the composition of the true measurements and their respective errors.
- 6) The pre-calibration of the inertial sensors using the algorithm proposed in [29]. Scale factors, non-orthogonalities and misalignment between triads were corrected.

After the ideal measurements' generation, the "null test" was carried out – the integration of ideal accelerometers' and gyroscopes' readings using the equations of motion and comparison of the result with the true values of velocity and position. For all generated trajectories, the null test was successfully passed – the error at the end of the integration did not exceed few centimeters. The parameters of the sensor errors during the simulations corresponded to real low-grade MEMS sensor (TDK MPU-9250 sensor was used as the reference). The simulation parameters are shown in Table I.

The Allan deviation plots for the real gyroscope and the simulated one are shown on Fig. 6. The duration of the data recording was equal to 24 hours both for real and simulated data. For the typical simulation duration (tens of minutes) the Allan deviation plots are similar.

##### B. Datasets

Three datasets were used as the inputs for algorithm testing:

- (D1) 500 routes and the associated sensors' measurements. The start and end points of each route were placed exactly on the one of the edges of the roadmap. The simulation parameters are specified in Table I.
- (D2) 100 routes that are analogues to the dataset (D1), but the start and end points of each route were outside the roadmap: in courtyards.
- (D3) 12 real logs. They were recorded while moving inside and outside the city. Their duration varied from 20 minutes to 1 hour. The error characteristics of the inertial sensor used for in-field data gathering campaign were on same level as the one used in the simulations. The sampling rate of the odometer was equal to 10 Hz, consumer GNSS receiver – 1 Hz.

The examples of the roads, that were selected for routing, are shown on Fig. 7. The intensity of the color of the road edge reflects the number of routes made through it. The time durations of the generated datasets and the travelled distances for dataset (D1) are shown on Fig. 8.

TABLE I: The simulation parameters.

|                                         |              |                                  |
|-----------------------------------------|--------------|----------------------------------|
| Minimum dataset duration                | $t_{min}$    | 600 s                            |
| Maximum vehicle velocity                | $v_{max}$    | 40 $\frac{km}{h}$                |
| Gyroscope                               |              |                                  |
| Sampling period                         | $\Delta t$   | 0.01 s                           |
| Bias                                    | $b_g$        | [0.1, 0.2, 0.7] $^{\circ}/s$     |
| Bias instability                        | $BI$         | 5.4 $^{\circ}/h$                 |
| Correlation time                        | $T_c$        | 600 s                            |
| Angle random walk                       | $ARW$        | 0.36 $^{\circ}/\sqrt{h}$         |
| Scale factor errors                     | $\Delta s$   | [0.02, -0.03, -0.015]            |
| Non-orthogonalities of the axes         | $\Delta m$   | [1.4, 0.8, 0.3] $^{\circ}$       |
| Triad misalignment w.r.t. accelerometer | $\epsilon$   | [0.3, 2.5, -1.7] $^{\circ}$      |
| Accelerometer                           |              |                                  |
| Sampling period                         | $\Delta t$   | 0.01 s                           |
| Bias                                    | $b_a$        | [0.1, 0.3, -0.2] $\frac{m}{s^2}$ |
| Bias instability                        | $BI$         | 0.06 mg                          |
| Correlation time                        | $T_c$        | 600 s                            |
| Velocity random walk                    | $VRW$        | 0.28 m/s/ $\sqrt{h}$             |
| Scale factor errors                     | $\Delta s$   | [0.01, 0.02, 0.025]              |
| Non-orthogonalities of the axes         | $\Delta m$   | [-1.1, 0.3, 0.6] $^{\circ}$      |
| Odometer                                |              |                                  |
| Sampling period                         | $\Delta t_o$ | 0.1 s                            |
| Scale factor error                      | $\Delta s$   | 0.02                             |
| RMSD error                              | $\sigma_o$   | 0.1 $\frac{m}{s}$                |
| GNSS receiver                           |              |                                  |
| Sampling period                         | $\Delta t_G$ | 1.0 s                            |
| RMSD horizontal error                   | $\sigma_h$   | 3.0 m                            |
| RMSD vertical error                     | $\sigma_v$   | 5.0 m                            |

### C. Processing methods

Each dataset (D1)–(D3) was processed under the following configurations:

- (M1) EKF with decalibrated IMU readings (without scale factors, non-orthogonalities and triad misalignment corrections).
- (M2) EKF with calibrated IMU readings (scale factors, non-orthogonalities and triad misalignment were corrected according to the algorithm [29]).
- (M3) RBPF using buildings' footprints.
- (M4) RBPF using roadmap.
- (M5) RBPF using both buildings' footprints and roadmap.

The value of  $d_{min} = 2.5 m$  was selected to process dataset (D1) under (M3)–(M5) configurations. We made so to estimate the potential navigation accuracy when perfectly known road network and buildings' footprints map are available. The value of  $d_{min} = 25 m$  was selected to process datasets (D2) and (D3) since we wanted to allow the off-road driving and account for road network representation inaccuracies.

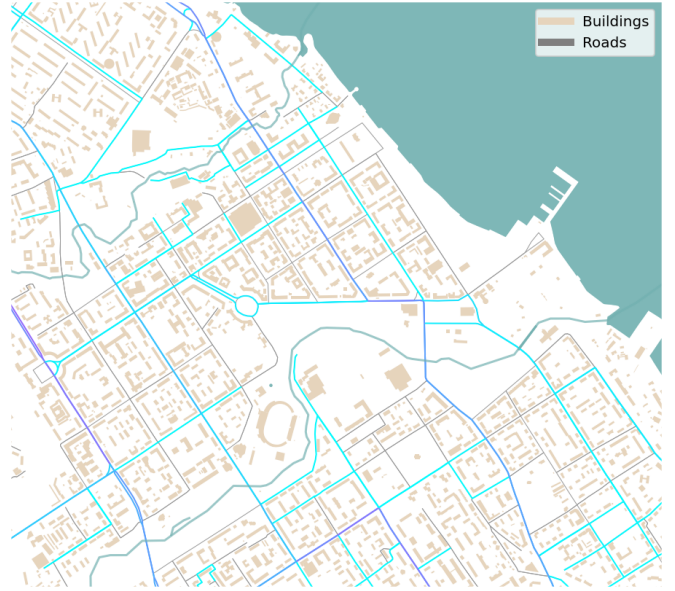


Fig. 7: The generated routes plotted on the city map. The color of the edge (from cyan to purple) – the number of routes plotted through the given edge. The buildings' footprints used for corrections are shown in light brown.

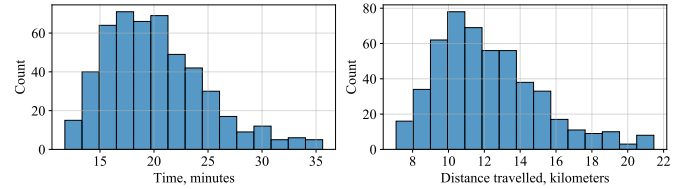


Fig. 8: The histograms of the time durations and route travel distances for logs from dataset (D1).

For each configuration the state was determined by (8). The GNSS corrections were switched off after 300 seconds of motion. The number of filters in the bank (or number of particles) in configurations (M3)–(M5) was equal to 500.

### D. Analysis of the results

All the datasets were processed on the eight-core personal computer in two-threaded mode. The CPU frequency was 2.3 GHz. The implementation was done in C++ language. The average ratio of the dataset experiment duration to the time spent for its processing on a PC was  $\frac{t_{log}}{t_{processing}} \approx 20$  for 500 particles. This means that the maximum number of particles suited for real-time processing is approximately equal to 10000.

Such a high number of potentially available particles can provide a dense coverage for a large initial position's uncertainty.

After 300 seconds from the start of each experiment the GNSS corrections were switched off and algorithm operated in dead-reckoning mode for datasets (D1) and (D3). For (D2) the GNSS corrections were not used at all.

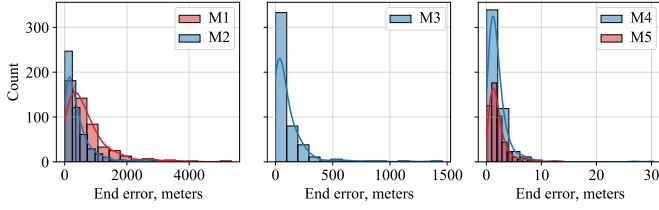


Fig. 9: The end error histograms of the horizontal position for configurations (M1)–(M5) for dataset (D1).

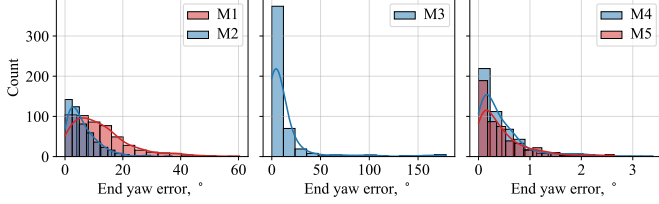


Fig. 10: The end error histograms of the yaw error for configurations (M1)–(M5) for dataset (D1).

In the next few paragraphs the processing results for datasets (D1)–(D3) are analyzed.

#### Dataset (D1)

After all logs were processed the descriptive statistics was calculated. The distribution of the end error of the horizontal position estimates is shown on Fig. 9. The end errors of the heading are shown on Fig. 10.

It can be concluded that the compensation of the instrumental errors using the pre-calibration can significantly reduce the position and attitude errors for traditional EKF (see the error histograms for configurations (M1) and (M2) on Fig. 9–10).

The usage of buildings' footprints without roadmaps (M3) in navigation leads to lower errors in comparison with EKF. However, these errors are higher than the corresponding ones in roadmap-aiding mode (M4). The reason is that in some areas of the urban environment the buildings' footprints are sparse. This leads to the formation of several clusters of possible vehicle's positions.

The best results are obtained for configuration (M5). Even when the vehicle moves along the roadmap, the buildings' footprints allow to reject, or null, the invalid hypotheses about the position. The final positioning error under configuration (M5) did not exceed 7 meters for 95% of cases, the final heading error did not exceed 3° for all tested logs. The maximum positioning error was equal to 12 meters.

The average experiment duration in dead-reckoning mode, as well as the average and standard deviations of the end errors are given in the Table II.

The estimates of the sensors' instrumental errors (gyroscopes' and accelerometers' biases, odometer scale factor error and GNSS receiver delay) reliably converged to their true values for all experiments.

In inertial navigation the problem of accurate initial alignment is well-known. With large initial heading error EKF

TABLE II: The end errors for configurations (M1)–(M5) in dead-reckoning mode\*.

| Configuration | Yaw error,<br>$\mu \pm \sigma, ^\circ$ | Position error,<br>$\mu \pm \sigma, m$ |
|---------------|----------------------------------------|----------------------------------------|
| (M1)          | $12.2 \pm 9.8$                         | $713 \pm 722$                          |
| (M2)          | $6.1 \pm 5.2$                          | $393 \pm 439$                          |
| (M3)          | $13.2 \pm 28.0$                        | $107 \pm 177$                          |
| (M4)          | $0.44 \pm 0.5$                         | $2.0 \pm 2.4$                          |
| (M5)          | $0.45 \pm 0.5$                         | $2.0 \pm 1.7$                          |

\* The average time spent in dead-reckoning mode is equal to  $\mu_t = 911$  seconds, its standard deviation  $\sigma_t = 280$  seconds.

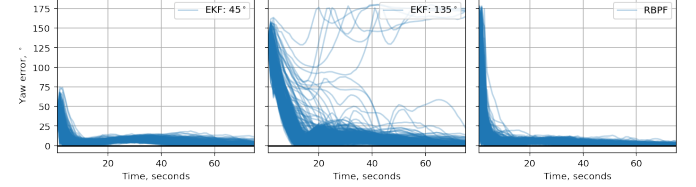


Fig. 11: The behavior of the heading error under different initial heading offsets for EKF and RBPF.

sometimes fails to converge. The poor initial alignment can deteriorate the subsequent position estimates. This effect degrades the positioning accuracy, and it is hard to completely eliminate it using the EKF.

To compare the heading convergence time using EKF and RBPF an additional series of experiments was carried out. All 500 logs were processed by two algorithms: using the EKF with different initial offsets ( $\pm 45^\circ$  and  $\pm 135^\circ$ ) and RBPF. The results are shown on Fig. 11. It should be noted that with  $\pm 45^\circ$  heading offset filter always converged to the true heading. However, there is a bulge in the heading estimate, when the heading error is not less than the threshold (see left plot on Fig. 11 from 20 to 30 second). RBPF is free from this issue and by 70-th second the heading error spread is almost half of that for the EKF.

Dataset (D2). No GNSS corrections at all.

The algorithm was additionally tested on quite challenging dataset (D2) – every route started and ended outside the roadmap, and there were not GNSS corrections at all. This mode can be seen as fully dead-reckoning – there were no global position corrections from the beginning. The initial position uncertainty  $\sigma_p$  was set to be 100 meters (the initial particles spread), initial heading was completely unknown. The convergence for the horizontal position and heading errors are shown on Fig. 12. Under the defined initial uncertainties the algorithm's global position and orientation estimates always converged to their true values. The convergence time did not exceed 100 seconds for the most part of the test cases. The maximum error did not exceed 50 meters (see Fig. 12).

Dataset (D3). Real data.

The algorithm performance in configuration (M5) was evaluated using the real data. The EKF processed trajectory with constant GNSS corrections was used as a reference one to assess the accuracy of operation in dead-reckoning mode.

The plots for the cumulative density functions for horizontal

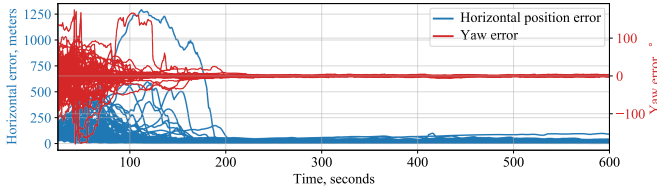


Fig. 12: The horizontal position and heading errors in full dead-reckoning mode since the start of the algorithm for dataset (D2). The initial position uncertainty was set to be 100 meters, initial heading was completely unknown.

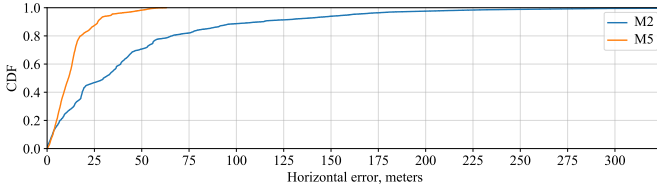


Fig. 13: Cumulative density function of the horizontal error for 12 real logs from dataset (D3). The errors are calculated for the time interval 0–600 seconds since GNSS corrections were switched off. Blue line – EKF, orange line – RBPF.

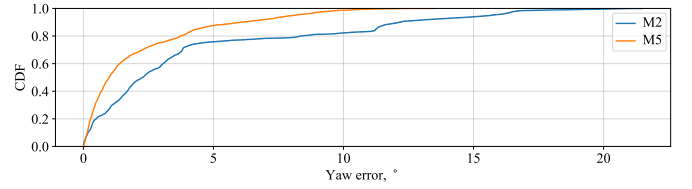


Fig. 14: Cumulative density function of the heading error for 12 real logs from dataset (D3). The errors are calculated for the time interval 0–600 seconds since GNSS corrections were switched off. Blue line – EKF, orange line – RBPF.

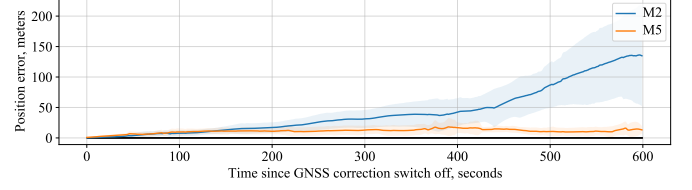


Fig. 15: The bounded position error in dead-reckoning mode for dataset (D3). Blue and orange lines –  $\mu \pm \sigma$  for EKF and RBPF during the real experiments.  $\mu$  – mean position error,  $\sigma$  – its standard deviation.  $\mu$  and  $\sigma$  are calculated based on the data from 12 real tests.

position error and heading error since the GNSS corrections were switched off and within 600 seconds after are shown on Fig. 13 and 14. The upper quartile of the position error for the RBPF is equal to  $\approx 15$  meters. Upper quartile of the heading error –  $\approx 2.0^\circ$ . For 95% of cases the positioning error did not exceed 25 meters, the maximum error – 50 meters.

It can be seen that the results of real experiments turned out to be worse than the results of simulations for dataset (D2). This is possibly caused by the following reasons:

- 1) Errors in the roadmap coordinates – sometimes the roadmap model's coordinates were significantly shifted w.r.t. their true values. Also, some roads are represented as one-lane, but in reality they were two or three lanes width. To compensate these inaccuracies  $d_{min}$  value was set to 25 meters while processing datasets (D2) and (D3).
- 2) Driving outside the roadmap. In real experiments the routes started and ended in the courtyards where the information about the graph was absent. During the exit from the roadmap a temporary increase in the positioning error was observed, caused by the transition of the RBPF to the new operation mode. After the transition, the growth of the error was limited again. This observation is verified by the analysis of the results for dataset (D2). The maximum error in this case did not exceed 50 meters as well.

It is interesting to note that, as well as for simulations, for all real experiments the growth of the position error turned out to be bounded on interval 0–600 seconds since GNSS switch off. This is shown on Fig. 15. The mean error for RBPF was not higher than 25 meters.

In the end we would like to present the navigation error

probabilities for three combinations: (D1)+(M5), where  $d_{min} = 2.5$  m, (D2)+(M5), where  $d_{min} = 25$  m, and (D3)+(M5), where  $d_{min}$  is also equal to 25 m. The results are presented on Fig 16.

## V. CONCLUSIONS AND FUTURE WORK

The robust dead-reckoning algorithm for the vehicle localization was developed.

The algorithm fuses the low-cost IMU, odometer, GNSS, buildings' footprints and roadmap data. It operates in global coordinates and estimates the vehicle's latitude, longitude, altitude and orientation, as well as the sensor errors. The algorithm is based on Rao-Blackwellized Particle Filter and is efficient in terms of computation load and achieved accuracy. The possible number of particles suited for the processing on a PC is approximately 10000.

The algorithm has been tested using the simulated and real data from low-cost inertial sensors. The results show that:

- 1) the localization error of the proposed algorithm is significantly lower than the corresponding error of the EKF: its mean value decreased from 400 to 2 meters after 15 minutes of operation in dead-reckoning mode;
- 2) the proposed algorithm improves the heading initialization time under large initial heading offsets: by the 70th second the heading error did not exceed  $7^\circ$ , and reduced by more than 2 times w.r.t. the maximum error for EKF ( $12.5^\circ$  under  $45^\circ$  heading offset);
- 3) the algorithm's position estimate converges even under large initial position and heading errors.

According to the simulation results on dataset (D1), the mean positioning error  $\approx 2$  meters, the final positioning error

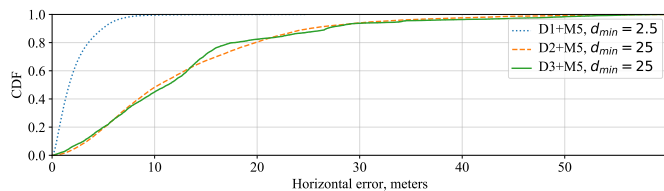


Fig. 16: Cumulative density function of the horizontal error for datasets (D1)–(D3) that were processed in mode (M5). The errors are calculated for the time interval 0–600 seconds since GNSS corrections were switched off.

did not exceed 7 meters for 95% of time after GNSS corrections switch off, the heading error was limited and did not exceed  $3^\circ$ . Given the detailed and accurate road network the lane-level precise navigation is possible using low-cost sensors in dead-reckoning mode. The usage of buildings' footprints allows to additionally decrease the position uncertainty by  $\approx 1$  m (from 2.4 to 1.7 meters).

The algorithm assessment on dataset (D3), gathered from real sensors, was performed both for the city roads and the highways. The positioning error did not exceed 25 meters for 95% of time between 0 and 600 seconds after GNSS corrections switch off. The mean error after ten minutes of operation in dead-reckoning mode was equal to 12 meters. This demonstrates the algorithm's applicability for scenarios when road networks are incomplete or have poor quality. The navigation accuracy degrades and becomes the road-level instead of the lane-level.

Further research will be devoted to the analysis of the roadmap errors affecting the positioning accuracy. Another research direction is to study the algorithm's feasibility to work without odometer corrections at all, utilizing only zero and turn velocity updates for the vehicle.

## REFERENCES

- [1] Erik Dahlman, Stefan Parkvall, and Johan Skold. *5G NR: The next generation wireless access technology*. Academic Press, 2020.
- [2] AD King. Inertial navigation-forty years of evolution. *GEC review*, 13(3):140–149, 1998.
- [3] KVH Industries. Guide to comparing gyro and imu technologies. <https://caclase.co.uk/wp-content/uploads/2016/11/Guide-to-Comparing-Gyros-0914.pdf>, 2014. Accessed: 2020-04-07.
- [4] V. V. Matveev. The engineering analysis of lapses of strapdown inertial navigational system. *Izvestiya Tula State University*, (9-2), 2014.
- [5] Henry Martin, Paul Groves, and Mark Newman. The limits of in-run calibration of mems inertial sensors and sensor arrays. *NAVIGATION, Journal of the Institute of Navigation*, 63(2):127–143, 2016.
- [6] Igor P Prihodko, Brock Bearss, Carey Merritt, Joe Bergeron, and Charles Blackmer. Towards self-navigating cars using mems imu: Challenges and opportunities. In *2018 IEEE International Symposium on Inertial Sensors and Systems (INERTIAL)*, pages 1–4. IEEE, 2018.
- [7] Aleksandr Mikov, Alexey Panyov, Vasily Kosyanchuk, and Igor Prihodko. Sensor fusion for land vehicle localization using inertial mems and odometry. In *2019 IEEE International Symposium on Inertial Sensors and Systems (INERTIAL)*, pages 1–2. IEEE, 2019.
- [8] Mohinder S Grewal and Angus P Andrews. Applications of kalman filtering in aerospace 1960 to the present [historical perspectives]. *IEEE Control Systems Magazine*, 30(3):69–78, 2010.
- [9] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE international conference on image processing (ICIP)*, pages 3464–3468. IEEE, 2016.
- [10] Guoquan P Huang, Anastasios I Mourikis, and Stergios I Roumeliotis. Observability-based rules for designing consistent ekf slam estimators. *The International Journal of Robotics Research*, 29(5):502–528, 2010.
- [11] Andrew W Long, Kevin C Wolfe, Michael J Mashner, and Gregory S Chirikjian. The banana distribution is gaussian: A localization study with exponential coordinates. *Robotics: Science and Systems VIII*, 265, 2013.
- [12] Axel Barrau and Silvére Bonnabel. The invariant extended kalman filter as a stable observer. *IEEE Transactions on Automatic Control*, 62(4):1797–1812, 2016.
- [13] Axel Barrau and Silvére Bonnabel. Three examples of the stability properties of the invariant extended kalman filter. *IFAC-PapersOnLine*, 50(1):431–437, 2017.
- [14] Lubin Chang, Jingbo Di, and Fangjun Qin. Inertial based integration with transformed ins mechanization in earth frame. *arXiv preprint arXiv:2103.02229*, 2021.
- [15] Jérémie Deray and Joan Solà. Manif: A micro lie theory library for state estimation in robotics applications. *Journal of Open Source Software*, 5(46):1371, 2020.
- [16] Rudolph Van Der Merwe et al. *Sigma-point Kalman filters for probabilistic inference in dynamic state-space models*. PhD thesis, OGI School of Science & Engineering at OHSU, 2004.
- [17] Joan Sola. Quaternion kinematics for the error-state kalman filter. *arXiv preprint arXiv:1711.02508*, 2017.
- [18] Zaher Zak M Kassas, Mahdi Maaref, Joshua J Morales, Joe J Khalife, and Kimia Shamei. Robust vehicular localization and map matching in urban environments through imu, gnss, and cellular signals. *IEEE Intelligent Transportation Systems Magazine*, 12(3):36–52, 2020.
- [19] Maan E El Najjar and Philippe Bonnifait. A road-matching method for precise vehicle localization using belief theory and kalman filtering. *Autonomous Robots*, 19(2):173–191, 2005.
- [20] Arnaud Doucet and Adam M Johansen. A tutorial on particle filtering and smoothing: Fifteen years later. *Handbook of nonlinear filtering*, 12(656-704):3, 2009.
- [21] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic robotics*. Kybernetes, 2006.
- [22] Fredrik Gustafsson. Particle filter theory and practice with positioning applications. *IEEE Aerospace and Electronic Systems Magazine*, 25(7):53–82, 2010.
- [23] Audrey Giremus, Arnaud Doucet, Vincent Calmettes, and J-Y Tourneret. A rao-blackwellized particle filter for ins/gps integration. In *2004 IEEE International Conference on Acoustics, Speech, and Signal Processing*, volume 3, pages iii–964. IEEE, 2004.
- [24] Thomas Schon, Fredrik Gustafsson, and P-J Nordlund. Marginalized particle filters for mixed linear/nonlinear state-space models. *IEEE Transactions on signal processing*, 53(7):2279–2289, 2005.
- [25] Gustaf Hendeby, Rickard Karlsson, and Fredrik Gustafsson. The rao-blackwellized particle filter: a filter bank implementation. *EURASIP Journal on Advances in Signal Processing*, 2010:1–10, 2010.
- [26] Karl Berntorp and Stefano Di Cairano. Particle filtering for automotive: a survey. In *2019 22th International Conference on Information Fusion (FUSION)*, pages 1–8. IEEE, 2019.
- [27] Paul D Groves. Principles of gnss, inertial, and multisensor integrated navigation systems, [book review]. *IEEE Aerospace and Electronic Systems Magazine*, 30(2):584–589, 2015.
- [28] Fredrik Gustafsson, Umut Orguner, Thomas B Schön, Per Skoglar, and Rickard Karlsson. Navigation and tracking of road-bound vehicles. *Handbook of Intelligent Vehicles*, pages 397–434, 2012.
- [29] Aleksandr Mikov, Sergey Reginya, and Alex Moschevkin. In-situ gyroscope calibration based on accelerometer data. In *2020 27th Saint Petersburg International Conference on Integrated Navigation Systems (ICINS)*, pages 1–5. IEEE, 2020.